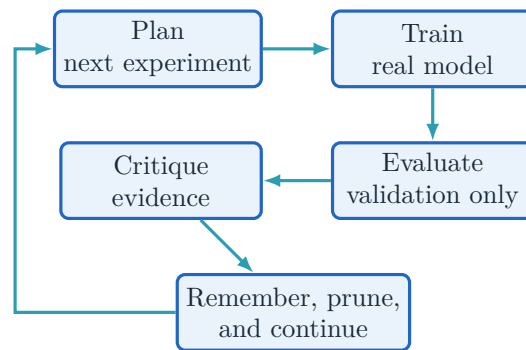


RecPilot

Evidence-Aware Autonomous Recommender Research Agent



Guo Xingchen | Brenda Tan | Junrong

Technical Report Draft – 31 August 2026

Report status. This draft contains implemented behavior, automated-test evidence, Person 1’s offline validation evidence, and a two-experiment validation-only autonomous smoke run. The official full-budget run and its one-time test finalization were still pending when this version was prepared; no final test metric is claimed.

Abstract

RecPilot is an autonomous machine-learning research agent for recommendation. Rather than executing a fixed hyperparameter sweep, it selects a research parent, proposes a legal experiment, trains a real recommender, evaluates validation GAUC and nDCG@5, critiques the evidence, and uses memory to decide what to try next. The system combines a typed research action space, an auditable best-first tree policy, structured criticism, compact research memory, duplicate prevention, evidence-aware hard and soft filtering, deterministic recovery, and strict time and iteration budgets. Validation and test responsibilities are separated: research sees validation evidence only, while test evaluation requires an explicit one-time finalization flag. A validation-only smoke run improved the organizer-defined Primary metric from 0.601469 for FM+BCE to 0.603963 for FM+BPR. This report distinguishes that autonomous trajectory from offline experiments and from the still-pending official finalized run.

1 Problem and Motivation

Recommender research exposes a combinatorial decision space: model family, training objective, feature construction, negative sampling, sequence design, ensemble composition, and optimization settings. A naive grid wastes expensive training runs, while an unconstrained language model can propose invalid, duplicated, or weakly justified experiments. Under a maximum of 50 experiments and six hours, each decision must produce useful evidence.

RecPilot reframes model development as a closed-loop research problem. The system is not merely a hyperparameter tuner and does not directly recommend content to users. It is an agent that improves the *research process*: it chooses experiments, executes them, interprets outcomes, stores lessons, and controls when to stop. Its design goals are:

- **Autonomy:** continue without a human selecting each experiment;
- **Scientific discipline:** separate observations from interpretations and avoid treating tiny gains as strong evidence;
- **Efficiency:** avoid duplicates, structurally impossible models, unsafe features, and repeatedly rejected mechanisms;
- **Auditability:** record every hypothesis, change, parent, metric, decision, error, token count, and intervention;
- **Evaluation safety:** keep test information away from planning and criticism.

2 System Architecture

The authoritative entry point is `scripts/run_agent.py`, backed by `src/techjam_agent/`. The similarly named root-level prototype is retained only for compatibility and is not used for official execution.

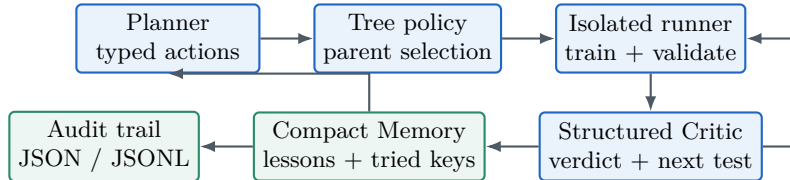


Figure 1: RecPilot’s research loop. Validation evidence returns to planning; test evaluation is outside the loop and occurs only during explicit finalization.

2.1 Planner and Research Action Space

The Planner operates over five closed action types: `TRY_MODEL`, `TRY_FEATURE`, `TRY_ENSEMBLE`, `TRY_SEQUENCE`, and `TUNE`. This compresses a large free-form search space into legal research families and concrete admissible changes. The candidate catalog covers FM, DeepFM, multitask and sequence variants, DCNv2, LightGBM, ensembles, ranking and hybrid objectives, feature families, and optimization settings. Configuration validation rejects incompatible combinations before training.

The deterministic Planner provides reproducible autonomous selection. An OpenAI-compatible LLM Planner is also implemented for exploratory use, with structured JSON parsing, retry/backoff, token accounting, sanitization, and deterministic fallback. If the LLM path fails, fallback keeps the same prior evidence rather than reopening directions already closed by artifacts or history.

2.2 Tree Policy and Lineage

Each experiment is a node with a stable identifier, parent, branch, configuration, result, and decision. Parent selection is distinct from global-best preservation. A child may improve over a weaker parent yet still be rejected if it fails to beat the global best. Conversely, a child from any branch can become champion if its validation Primary is higher.

The tree policy is a lightweight auditable best-first heuristic. It combines validation exploitation, exploration and novelty bonuses, runtime cost, repetition, failed-child evidence, and rejected-node evidence. It is **not MCTS**: there are no simulated rollouts, simulation phase, value backpropagation, or self-training from planned trajectories.

2.3 Runner, Critic, and Memory

Experiments execute in isolated subprocesses with a 900-second timeout. Errors, timeouts, non-finite metrics, and missing Primary values become structured failed records instead of terminating the entire research loop. Successful runs return validation GAUC, nDCG@5, Primary, best epoch, runtime, and a checkpoint.

The Critic outputs a JSON object containing observation, interpretation, confidence, verdict, delta, meaningful-improvement status, next test, and reasons. Its epsilon is 0.002:

Validation condition	Verdict	Meaningful improvement
Error, timeout, non-finite, or missing Primary	failed	false
$\Delta > 0.002$	promote	true
$-0.002 \leq \Delta \leq 0.002$	noise	false
$\Delta < -0.002$	reject	false

Checkpoint retention and scientific interpretation intentionally differ. The Controller keeps any candidate that strictly exceeds the global-best score, while the Critic requires a gain larger than epsilon to call it meaningful. Memory then compresses history into baseline, best observed, promising, uncertain, negative, and failed evidence, while retaining uncapped internal experiment keys for exact duplicate prevention.

3 Evidence-Aware Selection

RecPilot uses evidence before spending a training run. Evidence sources are validation-only JSON artifacts routed through a manifest; Markdown notebooks such as `TRY.md` are human research notes and are never parsed by the runtime agent.

3.1 Hard Blocks, Soft Stops, and Two-Pass Search

- A **hard block** represents a direction that cannot or must not run, such as an unavailable model, an invalid configuration, explicitly unsafe leakage, zero eligible rows, or a robust scoped negative supported by sufficient folds or seeds.
- A **soft stop** lowers priority for uncertain or weak evidence, including low but non-zero feature coverage, repeated generic failures, rejected mechanisms, high prediction correlation, or incomplete leakage evidence.

Selection uses two passes. The preferred pass excludes hard-blocked and soft-stopped candidates. If nothing remains, the relaxed pass permits soft-stopped directions but never permits hard blocks or exact duplicates. This avoids both wasteful repetition and premature search exhaustion.

3.2 Feasibility Signals

The feasibility layer supports feature coverage, measured runtime, pairwise prediction correlation, and a per-feature leakage registry. A validation-only coverage audit produced the following evidence for candidate-history features:

Feature	Coverage	Eligible / total	Preferred-pass effect
<code>prior_video_positive</code>	0.000304	38 / 124,909	soft stop
<code>author_positive_recency</code>	0.007269	908 / 124,909	soft stop
<code>previous_author_same</code>	0.005916	739 / 124,909	soft stop
<code>prior_video_count</code>	0.022809	2,849 / 124,909	remains admissible

Four target-rate feature groups use frozen train-only statistics with leave-one-out construction for training rows and do not read validation/test labels. However, their aggregate is not strict-past within the training window. They therefore remain honestly marked `leakage_safe=true`, `strict_past=false`, and `status=uncertain` rather than being silently treated as causal features.

4 Budgets, Convergence, and Evaluation Safety

4.1 Resource Controls

The maximum iteration count includes the baseline: 50 means one baseline plus at most 49 candidates. A monotonic global clock begins before the baseline and covers planning, training, evaluation, and reasoning. Before starting an experiment, the Controller reserves the full 900-second legal timeout; it refuses a run that could cross the six-hour boundary.

Convergence is tracked only from finite candidate comparisons. Three consecutive global-best expansions without improvement above epsilon indicate stagnation. Deliberate exploration of a weaker parent is neutral: it neither advances nor resets the streak. A breakthrough larger than epsilon resets the streak regardless of which branch produced it. This prevents tree exploration from creating a false convergence signal.

4.2 Validation/Test Isolation

The Planner and Critic receive validation metrics only. Evidence sanitization removes test keys, and validation evidence sources must explicitly state that test labels were not used. The normal research runner does not evaluate test data. Test evaluation and `submissions/final.csv` generation occur only when `-finalize-test` is passed to the same process that performs the official validation search. The current CLI cannot post-hoc finalize a champion from an earlier `logs/run_*` directory; documenting a separate finalization process would risk retraining and evaluating a different trajectory.

5 Experimental Evidence

5.1 Evidence Categories

We separate three categories to avoid overstating autonomy:

1. **Offline ML evidence:** model and feature studies conducted by the ML track;
2. **Autonomous smoke evidence:** a short validation-only trajectory produced by the agent;
3. **Official evidence:** the full-budget finalized trajectory, pending in this draft.

Evidence	Configuration	GAUC	nDCG@5	Primary	Status
Autonomous smoke baseline	FM + BCE, lr = 0.001	0.667133	0.535805	0.601469	reference
Autonomous smoke candidate	FM + BPR, lr = 0.0003	0.670594	0.537332	0.603963	promote
Offline validation champion	60% FM+BPR + 40% DeepFM+BCE	–	–	0.604713	offline only
Official finalized run	Pending	–	–	–	not executed

Table 1: Verified validation evidence available when this report draft was prepared. The offline champion is not claimed as an autonomous discovery. No final test score is reported.

The two-experiment smoke run completed in 84.04 seconds using the deterministic Planner. The candidate improved Primary by +0.002494, exceeding the Critic epsilon and receiving **promote**. The run made zero LLM requests and did not finalize test metrics. Its purpose was to verify the full Planner–Runner–Critic–Memory–Tree integration, not to replace the official search.

Offline studies found the 60/40 ensemble above and reported positive rolling evidence across three folds with mean Primary delta +0.001123 over FM+BPR. These results help define available mechanisms and priors, but remain distinct from what an autonomous trajectory has selected.

5.2 Engineering Verification

The pre-official-run branch passed 257 automated tests in 1.890 seconds. Tests cover structured Critic boundaries, memory compatibility, exact duplicate prevention, budgets, failure recovery, parent/global-best separation, exploration-safe convergence, action-space classification, hard/soft filtering, evidence schemas, replay parity, safe finalization, and fallback evidence retention. The test suite uses fake runners and synthetic metrics; its printed scores are not experimental results.

6 Research Foundations

RecPilot draws design inspiration from recommendation-agent literature without claiming to reimplement or reproduce any paper:

- **RecMind** motivates reviewing all previously explored states during planning. RecPilot operationalizes this idea through compact memory and distilled research patterns rather than zero-shot user recommendation [1].
- **TAIRA** separates manager planning from specialized execution and distills useful thought patterns from

successful and corrected trajectories. RecPilot is conceptually aligned through Controller/Runner separation and evidence reuse, but implements neither TAIRA’s interactive recommendation task nor its full Thought Pattern Distillation method [2].

- **SAPIENT** demonstrates strategic tree planning for conversational recommendation. RecPilot borrows the motivation for non-greedy exploration but deliberately uses a single-step best-first policy, not MCTS [3].
- **STARec** motivates deliberate rather than purely reactive reasoning. RecPilot’s structured Critic and evidence-conditioned next action are conceptually aligned, but it does not reproduce STARec’s user-agent cognition, anchored reinforcement training, or slow-thinking training procedure [4].

Full MCTS is impractical under the challenge budget because each rollout would require real model training. At the 900-second experiment ceiling, ten rollouts could consume up to 2.5 hours for one planning decision; two such decisions could consume up to five hours before planning, evaluation, and finalization overhead. RecPilot instead spends each training run on an experiment that produces real validation evidence.

7 Team Contributions

Three team members contributed across complementary offline-ML and autonomous-agent tracks.

Team member	Role	Primary contributions
Guo Xingchen (Person 1)	Recommender models and validation	Model-training pipeline covering FM, DeepFM, LightGBM, ensembles, Sequence DeepFM, and DCNv2; BPR negative sampling; initial agent and evidence infrastructure; analysis and offline validation.
Brenda Tan (Person 2)	Autonomous research decisions	Structured Critic; compact Memory and deduplication; budget and convergence controls; parent/global-best tracking; typed action space; hard/soft evidence and two-pass selection; feasibility routing; LLM hardening; evidence-aware fallback; replay parity; CLI safety; research-foundations documentation.
Junrong (Person 3)	Setup, tree-search, and integration	Repository and starter-kit setup; early FM-BPR/LightGBM agent integration; initial tree-search contribution. Assigned official finalized run responsibility remained pending at report preparation time.

Core files including `controller.py`, `proposals.py`, and `tree.py` have contributions from all three members. Training logic and feasibility producers have Person 1 and Person 2 as their respective primary contributors; process isolation is shared between Person 1 and Person 3. Detailed attribution remains available from Git history rather than personal email addresses in this public report.

8 Limitations and Next Steps

The most important limitation is procedural: this draft predates the official finalized run. Accordingly, it does not claim an official champion, final test metric, finalized checkpoint, submission validation, full-budget trajectory, or final resource accounting. Those fields must be filled directly from the official `summary.json` and artifacts, never from a notebook or a repeated test run.

Other limitations are technical. The deterministic candidate catalog is intentionally bounded; LLM proposals require external credentials; some target-rate features are not strict-past; runtime reservation is conservative; and the tree policy is heuristic rather than learned. Future work could add calibrated runtime prediction, multi-seed uncertainty, richer cross-dataset memory transfer, and a visualization of the auditable research tree.

Before submission, the team must execute the frozen latest-main command once, with explicit finalization, then record: best configuration and validation metrics, final test metrics, total experiments, stop reason, elapsed time, LLM request/token counts, manual interventions, failures and recoveries, compute resources, checkpoint path, submission path, and the complete trajectory.

References

- [1] Y. Wang et al., “RecMind: Large Language Model Powered Agent for Recommendation,” *Findings of NAACL*, 2024. <https://aclanthology.org/2024.findings-naacl.271/>
- [2] H. Yu et al., “Thought-Augmented Planning for LLM-Powered Interactive Recommender Agent,” arXiv:2506.23485, 2025. <https://arxiv.org/abs/2506.23485>
- [3] H. Du, B. Peng, and X. Ning, “SAPIENT: Mastering Multi-turn Conversational Recommendation with Strategic Planning and Monte Carlo Tree Search,” *NAACL*, 2025. <https://aclanthology.org/2025.naacl-long.133/>
- [4] C. Wu et al., “STARec: An Efficient Agent Framework for Recommender Systems via Autonomous Deliberate Reasoning,” *CIKM*, 2025. <https://arxiv.org/abs/2508.18812>